

Fold, Don't Stream: Recurrent Depth as a Storage-Residency Optimization for Consumer-Device LLM Inference

Gigyeom Ahn

Independent study, conducted in collaboration with an AI research assistant†

July 27, 2026 — preprint draft v3 (adds Metal execution of folded models and a measured ceiling on training-free folded quality; revised after a four-lens adversarial review)

Abstract

When a language model exceeds a consumer device's RAM, decoding falls off a *storage cliff*: every token forces the active parameters to be re-read from disk. We measure it directly on an Apple M1 Max (32 GB), where a dense 70B model at 4-bit reads 40–44 GB from SSD *per generated token* and decodes at ~ 0.02 tokens/s. Recurrent depth — executing a small stack of layers R times — is an established mechanism; we treat it as a lever against that cliff rather than as a parameter- or reasoning-efficiency one, and measure both what it buys and what it cannot.

Controlled studies at 15M and 42M parameters show that at equal resident weights *and equal training tokens* a looped transformer recovers 41% (dense) and up to 54% (Mixture-of-Experts) of the quality gained by doubling memory, reproduced on two hardware/precision stacks. Inside looped MoE, *sticky* routing — re-using the first pass's expert selection — matches free re-routing within seed noise while freezing expert traffic at top- k , so access locality is free. A graph-folding patch to llama.cpp then removes the cliff on the same laptop: $\sim 120\times$ for the dense 70B and $\sim 1.7\times$ for a 118B-A8B MoE, with zero swap. Scoping the loader to the folded working set puts that 118B checkpoint on the GPU, where 57.6 GB of weights decode at **34.4 tokens/s** with 25.2 GB wired — with degraded output.

We then bound that degradation. Targeted repairs take the folded 118B from perplexity 4.2×10^4 to 27.6 against 6.40 unfolded, and no further: a per-token least-squares solve over the physical layer's *entire* 256-expert set — a lower bound on any routing, router refit, or gating adapter — leaves 0.93–0.95 of the target residual, against a chance floor of 0.957 that any 256 directions in 3072 dimensions achieve. On the trajectory the fold produces, the physical layer's expert basis is barely more useful for its neighbour's routed function than a random subspace, so no gating-side repair closes the gap and quality must come from loop-native training. As a controlled endpoint, a *flattened twin* of a production looped model — function and output identical by construction — isolates residency as the sole cause of a $\sim 150\times$ speed gap.

1 Introduction

This work began as a failure report. Attempting to serve Qwen3.5-122B-A10B (4-bit, 78.6 GB) on an M1 Max with 32 GB of unified memory, we exhausted the software toolbox — prefetch-depth tuning, page pinning via `mlock`, an explicit expert cache, three speculative-decoding variants, reduced expert counts, thread sweeps — and none moved decode speed beyond 1.5 tokens/s. Instrumenting the router explained why: over roughly 300 token-steps, 75% of all 12,288 (layer, expert) slots were touched, so the "hot set" is the entire pool and no caching policy can help with a stock router (post-hoc router fine-tuning that concentrates expert reuse [12] could shift this boundary; we did not evaluate it). The binding constraint is not compute but *bytes per token*: an autoregressive step must read all active parameters, and once the model exceeds RAM those reads hit storage at ~ 2 GB/s instead of memory at tens-to-hundreds of GB/s.

Two mathematical escapes exist: read *less* (sparsity) or read the same bytes *more times* (amortization). This paper develops the depth-axis version of the second: a **recurrent-depth** (looped) transformer stores L unique layers and executes them R times, so the resident footprint is that of L layers while effective depth is $L\cdot R$. Prior work on looped transformers

frames the benefit as parameter efficiency [2,3] or test-time reasoning [4,5,6]; the only residency argument we could locate is a single paragraph about a 20 MB SRAM in MobileLLM [2]. The DRAM-vs-SSD version of the claim — *fold the stack so unique weights fit in RAM, and the storage cliff disappears* — appears never to have been designed for and measured.

Contributions. (1) *Controlled quality studies* at 15M scale with pre-registered kill-gates: at equal resident weights and equal training tokens, loops recover 41% (dense) / up to 54% (MoE) of the $2\times$ -memory quality gap, reproduced across M1 Max/fp32 and NVIDIA T4/fp16 (§3) — with a compute-matched control that bounds the claim to the equal-token regime (§3.2). (2) A previously unmeasured MoE design axis, the *loop-internal routing policy*, with the multi-seed finding that *sticky* routing matches free re-routing within seed noise at zero additional expert traffic (§3.3). (3) An R -sweep establishing saturating returns and a practical optimum at $R=3\text{--}4$ (§3.4). (4) *Systems evidence* that residency, not bandwidth engineering, is the binding variable: a small llama.cpp patch folds existing checkpoints at inference time and removes the storage cliff on a consumer laptop — $\sim 120\times$ for dense 70B, $\sim 1.7\times$ for a 118B MoE against an untuned streaming baseline, benchmarked against an aggressive-quantization baseline, and GPU decode of a 57.6 GB

checkpoint at 34.4 tokens/s — with a *flattened-twin* experiment on a production looped model: an identical-function pair in which residency alone separates 15.1 from 0.1 tokens/s (§4.1–4.4). (5) A *measured ceiling* on what folding an untrained checkpoint can be worth. Targeted repairs take a folded 118B MoE from perplexity 4.2×10^4 to 27.6 against 6.40 unfolded, and no further: under an oracle-routing lower bound the physical layer's entire 256-expert set leaves 0.93–0.95 of the target residual, against a 0.957 chance floor. On the trajectory the fold produces, its expert basis is barely more useful for its neighbour's routed function than a random subspace of the same dimension; solved against the unfolded model's activations the stack brackets 0.74–0.95. So a repair confined to the gating side of the executed fold has little to work with, and the systems mechanism needs loop-native weights (§4.5).

Scope. Quality conclusions are claimed at 15M scale (30.4 MB fp16) on TinyStories only; folding a checkpoint that was not trained to loop destroys output quality by design, and our folding experiments measure throughput physics; §4.5 measures how far training-free repair moves quality and where it stops. The two halves compose: loop-trained models supply quality (§3, [6,16]); folding supplies the residency arithmetic that makes them attractive on consumer hardware (§4). The flattened-twin experiment (§4.4) closes the gap between the halves — coherent, loop-native, and beyond-RAM in a single measurement.

2 Related work

Recurrent depth. Layer reuse dates to Universal Transformers [1] and ALBERT. MobileLLM [2] adopted immediate block-wise sharing for sub-billion on-device models; Relaxed Recursive Transformers [3] convert pretrained models into looped form with per-loop LoRA; Huginn [4] scales test-time compute by looping to depth 64; Mixture-of-Recursions [5] routes tokens to per-token recursion depths; Ouro [6] pretrains looped models on 7.7T tokens, with 1.4B×4 matching a dense 4B. MELT [7] shares KV across loops. Looped MoE itself is established: MoEUT [18] applies the shared-layer (Universal Transformer) form with MoE in both FFN and attention; LoopMoE [19] adds iteration-conditioned modulation and a parameter- and FLOP-matched looped-vs-vanilla MoE comparison; Hyperloop Transformers [20] loop only a middle block — the begin/middle/end partition our protocol also uses (§3.1) — and report the advantage surviving post-training quantization (a parameter-matched comparison axis, distinct from our memory-doubling recovery metric); LT2 [21] replaces quadratic attention inside the loop. Throughout this line the stated benefit is parameters, quality, or reasoning; storage I/O is not measured.

Tied experts. Closest to our §3.3, Tying the Loop [8] shares expert FFN weights across consecutive layers with per-layer

independent routers, showing that full re-routing inside a tied group is nearly free once attention is untied — consistent with our re-route arm. It does not examine fixing the routing across loops, nor does it measure expert-touch counts or bandwidth; its framing is total-parameter reduction. Our sticky-vs-reroute comparison and access-locality result occupy the gap.

Storage-tier inference. LLM-in-a-flash [9], AirLLM-style layer streaming, and FlexGen [10] stream a *non-looped* model's weights through a small memory; MoE offloading systems [11,12] exploit routing locality that, per our measurements, base models do not reliably provide. None combine streaming with layer reuse. SHARP [13] shares adjacent-layer pairs and reports 42% smaller storage and loading time on an iPhone — mechanism-adjacent, but pairs rather than loops, and its models fit in RAM; the beyond-RAM cliff is not at issue. A 2024 llama.cpp discussion [14] proposed runtime layer reuse but was never implemented or measured. Nanbeige4.2-3B [16] is, to our knowledge, the first production looped release (22 layers × 2), framed as capacity-without-parameters; we use it as our ecosystem probe.

Our claim is therefore narrow and, to the best of a multi-angle search (systems venues, citation graphs of [2–6,18–21], patents, community forums), unoccupied: *the loop × beyond-RAM × measured-storage-I/O triple*. That the looped-transformer space is by now crowded — shared-layer MoE [18,19], partial-loop architectures [20], loop-efficiency variants [7,21] — sharpens rather than weakens the observation: none of them reports storage I/O, beyond-RAM decode throughput, or a loop-internal routing-policy comparison. We flag the framing risk in this sentence explicitly: an intersection of three narrow conditions is trivially unoccupied by construction, so its emptiness is not by itself evidence of contribution. To be precise about what is and is not new — we claim a measurement and framing contribution, not a new mechanism. Layer reuse, KV sharing across loops [7], and storage-tier streaming [9–13] are each individually established; what we add is (i) the first direct instrumentation of the beyond-RAM decode cliff under looping on real checkpoints, including an identical-function control (§4), (ii) a previously unmeasured design axis inside looped MoE (§3.3), and (iii) the residency-vs-context tradeoff this combination implies (§5, Fig. 4). Readers who consider "the triple was open" insufficient justification should weigh (i)–(iii) instead.

3 Controlled quality studies

3.1 Protocol

A single GPT implementation expresses every variant by configuration only (middle-cycle sharing: first/last layers unshared — the begin/middle/end partition also used by Hyperloop Transformers [20]; input re-injection; loop-index embedding; pre-norm RMSNorm). Data is TinyStories with a 4k BPE vocabulary (480.7M train / 4.8M val tokens); training

uses AdamW (6e-4, cosine), an effective batch of 32,768 tokens, and a fixed seed so all arms consume identical batch sequences. Metrics: validation loss; for MoE additionally the mean number of unique experts touched per (token, layer-slot), accumulated across loops. Kill-gates were fixed before running. We define **recovery** = (small - loop)/(small - large): the fraction of the quality gained by doubling resident memory that the loop obtains for free. Residency equality throughout §3 refers to *weights*; KV grows with logical depth and is priced separately in §5.

3.2 Dense: equal-residency comparison

Three models trained on 200M tokens each: *small* (4 layers, depth 4, 30.4 MB resident at fp16), *loop* (the same 4 layers, middle block looped $\times 3$, depth 8, 30.4 MB), *large* (8 layers, depth 8, 56.1 MB). Table 1 shows the loop beating its equal-residency baseline decisively and recovering $\sim 41\%$ of the doubling gap, with the entire experiment reproduced on independent hardware and precision. The sensitivity control passed (large-small = 0.091 nats), and loop training cost $1.9\times$ the baseline's wall-clock — the expected payment in compute. Recovery figures throughout §3 are single-seed; substituting the across-seed loss variance measured in §3.3 (± 0.0035 nats) into the 0.091-nat denominator implies roughly ± 4 percentage points of uncertainty on any individual recovery number — a band that also spans the late R-sweep increments of §3.4.

That payment is a genuine confound: loop arms consume more training FLOPs than their equal-token baseline, so part of the recovered gap could reflect "more compute spent," not "looping specifically." We ran the compute-matched control: training *small* from scratch to the loop arm's approximate FLOP budget (400M tokens, $\sim 2\times$) reaches val loss **1.4936** — better not only than loop (1.5577) but than *large* at 200M tokens (1.5038). At this scale and data regime, when compute rather than data is the binding constraint, spending it on more tokens for the resident-sized model beats looping. The quality claims of this section are therefore *equal-token* results: looping converts spare compute into recovered quality when the token budget is fixed (data-limited or epoch-limited settings). We state plainly that in our own setup this regime was imposed rather than intrinsic — 480.7M training tokens were available and the 200M budget was a design choice — so the equal-token regime's practical relevance rests on scales where data genuinely binds, and the literature's large-scale loop results [6] train far past our budgets. The confound is training-side only: at decode time the extra loop FLOPs are near-free on the bandwidth-bound hardware this paper targets — §4's own measurement (0.02 tokens/s streamed, saturated by ~ 44 GB/token of SSD reads, not by compute) shows decode FLOPs are not the bottleneck being traded against, so the residency/throughput claim (§4) is unaffected.

Table 1. Dense equal-resident-weights study (val loss, 200M tokens). Both columns run the same configuration, the same seed (1337), and the same tokenized data files — hence the same batch order — on independent hardware/precision stacks; this is an implementation-and-hardware reproduction, not a statistically independent replication (seed variance is quantified separately in §3.3). Under a shared data path, 3–4-decimal agreement is the expected outcome; the exact 4-decimal match for *small* is rounding at published precision, not a copied number.

Model	Resident	Depth	M1 Max fp32	Colab T4 fp16
small	30.4 MB	4	1.5950	1.5950
loop (R=3)	30.4 MB	8	1.5577	1.5573
large	56.1 MB	8	1.5038	1.5045

3.3 MoE: the loop-internal routing policy

Real large models are MoE, and looping an MoE poses a question with direct bandwidth consequences: when the block runs again, does the router choose experts *again* (re-route), or re-use the first pass's selection (*sticky*)? Re-routing can only grow the per-token expert union — the quantity that killed caching in our 122B measurements — while sticky freezes it at top-k. We trained four 42M-parameter MoE models (16 experts, top-2, d=384, 100M tokens): a no-loop baseline, both loop policies at R=3, and a $2\times$ -memory upper bound.

Table 2. Looped MoE at equal resident weights (best val loss; touches = mean unique experts per (token, layer-slot) accumulated over loops, averaged over all four MoE modules — the unshared pre/post blocks run once and are therefore capped at top-k=2, so this column's ceiling is 4, not 6). All four rows are from the Colab (T4/fp16) stack, as in Table 1's second column; the repository also ships two local M1/fp32 runs of the same configurations from a different cycle (*moe-small* best val 1.7611, and a *moe-loop-st* run truncated at step 2500), which are not comparable to these rows and are not the source of any figure here.

Model	Val loss	Touches	Recovery
moe-small (no loop)	1.7655	2.000	—
moe-loop, sticky	1.7129	2.000	49.7%
moe-loop, re-route	1.7081	2.369	54.2%
moe-large ($2\times$ mem)	1.6596	2.000	100%

Three findings (Fig. 3). First, loops work at least as well on MoE as on dense — recovery is *higher* (49.7–54.2% vs 41%). Second, the policy comparison. Table 2's single-seed run shows sticky trailing re-route by 0.28% in loss (pre-registered abandonment threshold: 2%), but single-seed determinism is not statistical robustness, so we replicated the comparison across $n=3$ seeds: the sticky-re-route difference is **+0.0032 $\pm$ 0.0035 nats** — statistical parity. An earlier draft read Table 2 as "sticky retains 92% of re-route's gain"; we retire that framing (the gap it quantified is itself within seed noise) in favor of a simpler and stronger statement: **at $n=3$, cross-loop routing freedom buys nothing measurable, so the bandwidth-optimal policy — sticky, touches frozen at top-k — is free.** One confound must be stated with it: our load-balance auxiliary loss is computed in the same branch that a sticky reuse skips, so the sticky arm accumulates one auxiliary term per looped module per forward and the re-route arm accumulates two. The arms therefore differ in balance

regularization as well as in routing, and the comparison — including the touch figure below — is of the two policies as implemented rather than of routing freedom in isolation. The loop-vs-baseline gap (~ 0.055 nats) is an order of magnitude above the seed noise and stands. Third, even free re-routing touches only 2.74 of a possible 6 experts inside the looped block (Table 2’s 2.369 averages all four MoE modules, two of which run once and are capped at top- $k=2$; the looped-block figure is $(4 \times 2.369 - 4)/2$): trained routers are $\sim 81\%$ self-consistent across loops, which suggests why constraining them fully costs nothing detectable. Consistent with [8], whose independent per-layer routers are our re-route arm’s analogue; looped and shared-expert MoE architectures exist [8,18,19], but none compares loop-internal routing policies or instruments expert touches — the sticky axis and the touch metric are, to our knowledge, first measured here. A terminology note: *sticky* in [12] denotes token-axis routing consistency (a training loss penalizing expert switches between adjacent tokens); ours is loop-axis consistency at inference. The two axes are orthogonal and composable.

3.4 How far can R go?

Sweeping $R \in \{1,2,3,4,6\}$ at 200M tokens each (Fig. 1): recovery rises $0 \rightarrow 27.6 \rightarrow 41.7 \rightarrow 48.5 \rightarrow 51.7\%$, with per-step increments roughly halving (27.6, 14.1, 6.8, then 1.6/step). Returns saturate toward $\sim 53\text{--}55\%$; the log-growth hypothesis is rejected, and the practical optimum is $R=3\text{--}4$ ($R=6$ pays 40% more compute for +3.2 points). The ceiling matches the capacity intuition: loops buy sequential depth, not parameter storage; knowledge capacity remains that of the unique weights [15]. Reaching the remaining half of the gap requires the knowledge axis (more unique parameters, retrieval, or memory layers) — loops and those techniques compose rather than compete. (The compute-matched caveat of §3.2 applies to the whole sweep: these are equal-token comparisons.)

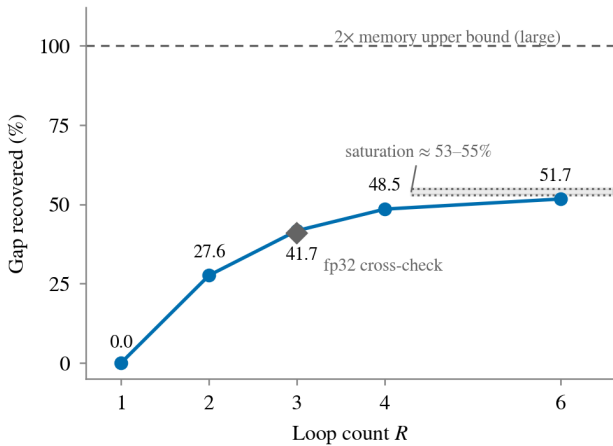


Figure 1. Fraction of the 2 $\times$ -memory quality gap recovered by looping, vs loop count R (dense, 15M scale, 200M tokens/run). Increments halve each step; returns saturate near 53–55%. The $R=3$ point is cross-checked on an independent fp32 stack (diamond).

4 Systems: removing the storage cliff

4.1 A graph-folding harness for llama.cpp

To measure the residency claim on real checkpoints we patched llama.cpp (~ 300 lines over two architectures): with `LLAMA_FOLD_K=K`, the build loop keeps its logical depth but takes every layer’s *weights* from physical layer $i \bmod K$. KV-cache slots, positions, and callbacks remain logical; weight-coupled structure follows the physical layer (head counts, dense-vs-MoE branch), and an assertion enforces sliding-window phase alignment (K must be a multiple of the SWA period). Only the first K layers’ file pages are ever touched, so with lazy mmap the resident set shrinks by K/n_{layers} without touching the GGUF or the loader. The full residency budget must include the KV cache, which folding does *not* shrink (it scales with logical depth): for the 70B fold below, 21.98 GB weights — the non-block tensors (token embeddings and output projection, 1.45 GB) never fold, so folded weights are not file size/ R — + ~ 1.3 GB KV (ctx 4096, fp16) + ~ 1 GB buffers ≈ 24.3 GB against $\sim 24\text{--}26$ GB usable — the fold factor is chosen so this *total* closes, and at $R=2$ it closes only near the top of that range. Measurement hygiene on macOS proved nontrivial (mandatory `--no-repack`; load-time prefetch disabled; a watchdog and a swap-delta guard); we record the failure modes in Appendix A.

§4.5 additionally uses an *expert-only* fold (`LLAMA_FOLD_EXPERTS=R`): the mixer — attention or gated delta-net, together with its norms and KV projections — keeps its own logical layer’s weights, and only the FFN block (router, routed experts, shared expert, post-attention norm) is taken from the physical layer. Because experts are 53.3 of Laguna’s 57.6 GB, this still folds 92.5% of the bytes while leaving the sequence-mixing path untouched, and unlike the cyclic map it admits per-layer exceptions — individual layers can be *pinned* to themselves, which §4.5 shows is necessary. Two further knobs, used only as ablations there, keep the shared expert and the FFN pre-norm on the logical layer, and scale the repeated layers’ routed branch by a constant γ .

4.2 Results: the cliff, measured and removed

Table 3. Folding existing checkpoints on an M1 Max 32 GB (CPU, 8 threads; 32 generated tokens per run, except the Laguna streamed baseline at 8; quality of the folded rows is measured separately in §4.5 — these checkpoints were not trained to loop). SSD bytes are page-in deltas for the whole run. Folded rows are means over $n=3$ repetitions; $\pm$ is sample std. Because weight-coupled structure follows the physical layer (§4.1), the Laguna fold at $K=16$ maps logical layers 16 and 32 onto physical layer 0, whose FFN is the leading dense block (`n_layer_dense_lead = 1`, `ff = 12288`): two of 48 layers execute a dense FFN in place of a 256-expert MoE block. Resident bytes and throughput are unaffected, but the incoherence of the folded rows cannot be attributed to looping alone. The final row is *not* a backend swap of the row above it: it uses the expert-only fold with three pinned layers and a logical shared expert (§4.1, §4.5), whose unique-weight set is correspondingly larger (25.0 GB vs 18.7 GB, both summed per tensor from the GGUF). Its residency column reports wired GPU memory rather than RSS, and its SSD column is left blank because we did not instrument the one-time load-time page-in for it; unlike the streamed rows it is not a per-token quantity, since the weights are copied into device-private buffers once and never re-read. Its throughput is the mean and sample std of three repetitions *within one llama-bench process*, measured at `-p 0 -n 32` (a near-empty KV cache) immediately after two perplexity evaluations; the CPU rows' $n=3$ are separate process launches with per-launch page-in accounting, so the two $\pm$ figures are not the same quantity and the Metal row does not sample the across-launch page-cache variation Appendix A(6) describes. That row also measures the *decode* path only: batched prefill in this configuration returns NaN (§4.2, Appendix A(11)). Because the configurations differ in fold map, unique-weight set, repair knobs, backend and KV occupancy, 34.41 and 2.37 are not on the same axis and should not be read as a backend speedup.

Configuration	Peak RSS	SSD read	Swap	Decode t/s
Llama-3.3-70B Q4 (42.5 GB), streamed	21 GB	1298 GB	0	~ 0.02
folded R=2 (21.98 GB unique)	21 GB	31–33 GB	0	2.40 ± 0.10
IQ2_XXS (17.8 GB), streamed, coherent	20 GB	30 GB	0	1.7
Laguna-S-2.1 118B-A8B (57.6 GB), streamed	22 GB	37 GB	0	1.4
folded R=3, K=16 (18.7 GB unique)	15–20 GB	18–20 GB	0	2.37 ± 0.42
folded R=3 + repair, Metal (25.0 GB unique)	25.2 GB wired	—	0	34.41 ± 0.74

The dense cliff is severe and its removal dramatic (Fig. 2): streamed, the 70B model re-reads the active weights nearly in full at every token — 355 GB over an 8-token run (44 GB/token) and 1298 GB over a 32-token run (41 GB/token; $31\times$ the 42.5 GB file) — and decodes at ~ 0.02 t/s (0.019: 32 tokens in 1713 s of wall-clock); folded so that unique weights fit in RAM, cold reads happen once (31–33 GB) and decoding runs at 2.40 ± 0.10 t/s — $\sim 120\times$, from a single environment variable, with zero swap. The streamed baselines here are untuned lazy-mmap streaming, not an optimized reader: sequential bandwidth (~ 2 GB/s) bounds an ideal double-buffered streamer at ~ 0.047 t/s for this file, so against the strongest streaming baseline the multiplier would be roughly $50\times$ — the cliff survives; its measured depth is baseline-dependent. The MoE cliff is inherently milder (only

active experts are read per token), so folding yields $\sim 1.7\times$; its benefit there is full residency of the active pool. (An early single run read 3.8 t/s; repetition attributes the excess to page-cache state — Appendix A(6) — and we report the $n=3$ mean.) The CPU rows deliberately hold the backend fixed and are floors, not ceilings. In v2.2 the Metal path was an open engineering item: offloading a folded model registered *all* mapped tensors as wired GPU memory, exceeding physical RAM (Appendix A(4)). Two changes close it — the loader no longer creates the tensors a folded graph never reads, and the remaining mmap-backed weights are copied into device-private buffers per tensor instead of the whole mapping being wired — after which the folded Laguna decodes at 34.4 ± 0.7 t/s ($n=3$) with 25.2 GB wired, and does so correctly. Scoring the same graph at matched batch, Metal gives 27.17 against 27.27 on CPU — a 0.4% backend divergence; the remaining gap to the 27.59 we report elsewhere is the batch, not the backend (the CPU number itself moves from 27.59 at ubatch 512 to 27.27 at ubatch 1). The throughput is therefore real computation rather than fast garbage. A 118B-A8B checkpoint whose 57.6 GB cannot be resident at all on this machine decodes on its GPU. We do not report a cliff multiplier for that row, because we did not re-measure the streamed baselines on Metal; all multipliers in this section remain CPU multipliers. One limit bounds the claim to the decode path: batched prefill in this configuration returns NaN. Sweeping the fold factor at ubatch 512 orders the failure by resident size — finite at $R=6$, $R=5$ and $R=4$ (15.0–17.6 GB of weights), finite at $R=3$ without the shared-expert knob (22.0 GB, perplexity 3.8×10^4), NaN at $R=3$ with it (22.4 GB), NaN for the deployed pinned configuration (25.0 GB), and an explicit `kIOGPUCommandBufferCallbackErrorOutOfMemory` at $R=2$ (31.3 GB) — while every one of these decodes correctly at batch 1. This device reports a 26.8 GB recommended maximum working set, and the sharpest evidence for that being the operative constraint is the 22.0 GB/22.4 GB pair: 0.4 GB of extra weights flips a finite result to NaN, which a kernel defect would not do. We nonetheless report this as an ordering, not a diagnosis. We did not raise `iogpu.wired_limit_mb`, because raising it far enough to admit this prefill would leave too little for the rest of the system to run — which is itself the point: on a 32 GB device the working-set limit is a deployment ceiling, not a conservative default to be tuned away. Folding therefore buys GPU decode of a model that could not be resident at all, and moves the binding constraint from SSD bandwidth to that ceiling during prefill. What these runs still do *not* show is quality: folded outputs are degraded, as expected for checkpoints never trained to loop. §4.5 measures how far that can be repaired without training, and shows the remaining gap is representational rather than an engineering deficit — quality must come from loop-native training (§3, [3,6,16]).

A skeptical reading of Table 3 asks why folding is necessary at all when aggressive quantization already shrinks a 70B checkpoint toward the budget with no architectural change. We ran that baseline directly: IQ2_XXS (~ 2.06 bits/weight, 17.8 GB) streams at 1.7 t/s with ordinary SSD traffic (30 GB) and *coherent* output. Our position is not that folding beats quantization on today's off-the-shelf checkpoints — for those, aggressive quantization is the practical default, and where it reaches usable RAM on its own, folding adds nothing. Folding's value is prospective and orthogonal: (a) for loop-native models (§3, §4.3–4.4) it delivers residency without the ~ 2 -bit quality tax, which post-hoc quantization of a non-looped model cannot match at equal bits; (b) quantization and folding compose — a folded model can itself be quantized ($R \times Q$ compounding), which we flag as the natural next measurement rather than an answered question.

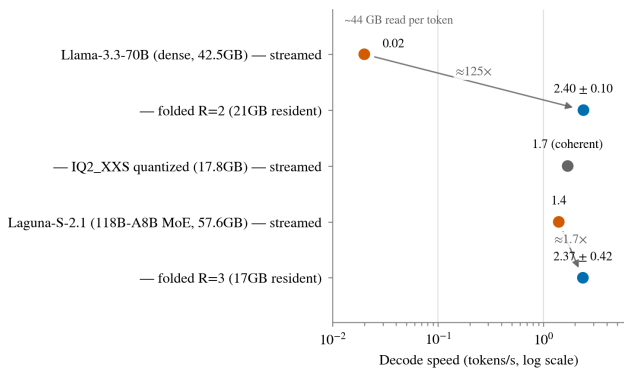


Figure 2. Decode speed before/after folding on the same laptop (log scale). Dense models suffer the full cliff (~ 44 GB re-read per token) and gain $\sim 120\times$; MoE reads only active experts and gains $\sim 1.7\times$. The aggressive-quantization baseline (IQ2_XXS, gray) is the practical choice for non-looped checkpoints: slower than folding but coherent.

4.3 Ecosystem check: a production looped model

Folding untrained checkpoints shows the physics; loop-native models show the product. Nanbeige4.2-3B [16] (22 unique layers executed twice; 2.57 GB at Q4) runs on the same laptop via the vendor's llama.cpp branch: output is fully coherent (a reasoning-mode code answer) at **49.4 tokens/s** decode / 115 t/s prompt on Metal. The mechanism validated at 15M scale in §3 is already shipping at 3B scale, and consumer inference of such models is practical today; llama.cpp mainline support is in review [17].

Comparability note: 49.4 t/s and Table 3's 2.4 t/s are not on the same axis and should not be read as "looping is $20\times$ better here." Nanbeige is a ~ 3 B, Q4, loop-native checkpoint that fits in RAM outright (2.57 GB) on the Metal backend — its number is normal small-model decode speed, not cliff removal. Table 3's figures are 70B/118B checkpoints never trained to loop, folded post-hoc purely to test storage physics, on a deliberately fixed CPU backend. The rows demonstrate different claims — "loop-native models already ship" vs "folding removes the cliff on arbitrary checkpoints" — and are adjacent only as complementary ecosystem evidence.

4.4 The flattened twin: identical function, different residency

The strongest objection to Table 3 is that its folded arms produce degraded output, so speed and quality are never demonstrated together. We close this with an identical-function pair built from the production model above. Nanbeige4.2-3B executes 22 physical layers twice (44 logical); we constructed a *flattened twin* by duplicating the 22 layers into a 44-layer non-looped GGUF (Q8_0: 4.43 GB folded, 7.78 GB flattened) and patched the inference graph so the model's loop-boundary normalization is applied at the same physical positions in both forms. Nanbeige inserts an extra RMSNorm (re-using the output norm) at each loop boundary, gated on loop count; a naive flatten silently drops it and changes the function. Our patch reproduces it at a configurable physical stride, restoring exact equivalence. The two checkpoints compute the same function: greedy decoding produces token-identical output. Unconstrained, both decode at an identical 15.6 t/s (CPU, 8 threads) — the pair differs by no compute. We then pinned 19 GB via an mlock ballast, sized to leave a ~ 7.5 GB window (the pre-ballast free-like reading was 27.1 GB; we did not re-measure after pinning, so the effective window is bounded by the harness target rather than observed): the folded form fits, the flattened does not. Result: folded **15.1 t/s** (10 GB SSD reads); flattened **0.1 t/s** with **231 GB** of SSD reads over 32 tokens, zero swap throughout — a $\sim 150\times$ gap in which architecture, weights, function, output, and hardware are all held equal and only residency differs. (The same baseline caveat as §4.2 applies: 0.1 t/s is lazy-mmap streaming; an ideal sequential streamer of the 7.78 GB file at ~ 2 GB/s would be bounded at ~ 0.26 t/s, i.e., a $\sim 60\times$ gap against the strongest streaming baseline.) This is Table 3's cliff with the quality confound removed by construction: a coherent, loop-native, beyond-RAM model whose speed difference is attributable to residency alone.

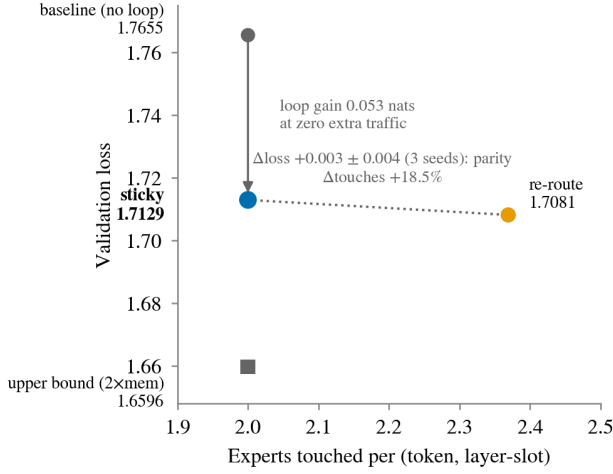


Figure 3. Looped-MoE routing policy as a quality/traffic trade (§3.3). Sticky routing sits at baseline traffic (2.000 touches) at quality statistically indistinguishable from free re-routing ($\Delta = +0.003 \pm 0.004$ across 3 seeds); free re-routing itself uses only 2.369 of a possible 4 experts averaged over all four MoE modules — the two unshared blocks run once and cap at top-k=2 — which is 2.74 of a possible 6 inside the looped block itself (§3.3).

4.5 How far training-free repair goes, and where it stops

Table 3 reports no quality for its folded rows because the outputs are incoherent. That is an observation about one configuration, not a bound, and it invites two distinct questions: how much of the loss is an artifact of *how* we fold, and how much is intrinsic to folding weights that were never trained to be shared? We answer both on Laguna-S-2.1 (48 layers, 256 routed experts plus one shared expert, IQ4_XS), scoring perplexity on the first two chunks of wikitext-2 (1024 tokens) at context 512, on the CPU backend so that the backend is not a confound. All perplexities below are on that fixed 1024-token window and move with it — at four chunks the unfolded model reads 7.18 and the best folded configuration 36.96 — so they are comparable to each other and not to other papers' wikitext numbers.

What repair buys. The naive expert-only fold — logical layer il takes its whole FFN block from physical layer wl — is catastrophic: perplexity rises from **6.40** unfolded to **1308** at $R=2$ and 4.2×10^4 at $R=3$. Three targeted changes recover most of it. (i) The *shared expert* must stay on the logical layer. Every token passes through it, so folding it applies a mismatched transform to the entire residual stream rather than to the 10-of-256 slice the router selects; keeping it logical alone takes $R=2$ from 1308 to **21.8**. This is the one repair with a strong interaction, and it is worth stating because it inverts: applied alone at $R=3$ it is *catastrophic* ($4.2 \times 10^4 \rightarrow 5.6 \times 10^6$), and only becomes the dominant lever once the late layers are pinned. Two wrong things can be better than one right one, and we would not have found the ordering by testing the repairs independently. (ii) The *final layers* must be pinned to themselves. At $R=3$, pinning layer 47 alone is worth a factor of four ($4.2 \times 10^4 \rightarrow 1.0 \times 10^4$), while pinning two *early* layers (1 and 2) instead changes nothing (3.8×10^4) — consistent

with the last block feeding the unembedding directly. We first read this as a position effect rather than a capacity one, but the control does not separate them: Laguna's UD quantization is non-uniform — layers 46–47 carry Q6_K and IQ4_XS against IQ3_S elsewhere — so pinning 47 restores both position and about 35% more bits, while the early-layer control restores neither. We report the ordering, not the mechanism. (iii) A scalar γ damping the repeated layers' *routed* branch has an interior optimum at $\gamma = 0.5$, worth a further $46.5 \rightarrow 27.6$. Fitting γ per layer by closed-form least squares does *not* beat the global constant (36.4, or 27.7 after a uniform rescale), which is the first indication that the residual error is not a gain mismatch. As a control on (i) we moved the FFN pre-norm to the logical layer, which is the symmetric intervention; it is *worse* (1465 vs 1308), so the two normalizations are not interchangeable and the shared-expert result is not a generic "keep more things logical" effect.

What repair cannot buy. Perplexity 27.6 against 6.40 is still unusable, and the failure mode is qualitative rather than marginal: greedy decoding emits one well-formed sentence and then repeats it verbatim, identically at perplexity 46 and at 27.6, so the metric and the usability have decoupled. To decide whether further engineering could close the gap, we measured an *achievability ceiling*. Fix a logical layer il folded onto wl . Because the shared expert is kept logical, the routed branch is required to produce $R = \text{FFN}_{il}(X) - \text{shexp}_{il}(X)$. Under `expert_weights_norm` the set that branch can reach is a scaled convex hull of layer wl 's 256 expert outputs, which is contained in their span; solving unconstrained least squares over all 256 experts *per token* therefore yields a strict lower bound on the residual attainable by any routing whatsoever — including an oracle router, a refitted router, or any gating-side adapter. Inputs X are taken from the folded model's own trajectory. That choice is not neutral, and we report both sides of it below.

Table 4. Achievability ceiling for the expert-only fold on Laguna-S-2.1. Values are relative residuals — residual norm divided by the norm of the target R ; lower is more reachable. The ceiling is per-token unconstrained least squares over all 256 experts of the physical layer wl , a lower bound no routing can beat. The two ceiling columns differ only in which trajectory supplies X : the folded model's own activations, or the unfolded model's. The chance floor is what *any* 256 directions in 3072 dimensions achieve by dimension counting, $\sqrt{(1-256/3072)}$; an i.i.d. Gaussian basis measures 0.957 empirically. The count of 256 is the right one to use: the per-token expert-output matrix is effectively full rank (condition number 3.2–7.7, and 248–250 of 256 singular directions needed to carry 99% of its energy), so the experts do not collapse onto a low-dimensional subspace that would make this floor too generous — recomputing it from the 99% rank gives 0.9584 against 0.9574. "Validation" repeats the identical solve against layer il 's *own* experts, where the true solution is feasible and the residual must vanish; that it does to three decimals checks the expert slicing, the dequantization and the solver for mutual consistency, but it is not an external check — target and basis come from the same forward code, so an error shared between them would not appear. We verified that forward pass against llama.cpp's graph construction by inspection instead. $T=128$ tokens throughout, except the re-pairing sweep below which is $T=64$; as executed (not solved) the folded routed branch's residual exceeds 1.0, i.e. it is worse than emitting nothing.

Logical il	Physical wl	Ceiling, folded X	Ceiling, unfolded X	Validation (il)
5	4	0.925	0.778	0.000
24	22	0.933	0.738	0.000
41	40	0.953	0.951	0.000
<i>chance floor</i> (any 256 directions in 3072)		0.957	0.957	—

Read against the floor, the result is narrower than the raw number suggests and sharper where it holds. On the folded trajectory the ceiling is 0.93–0.95 against a chance floor of 0.957: oracle routing over the entire expert set recovers 9–14% of the routed branch's energy against 8.3% for an arbitrary subspace of the same dimension (30–38% of its norm against 29%). On the fold *as executed*, then, layer wl 's expert basis is only marginally more useful for layer il 's target than 256 random directions.

But the folded trajectory is itself collapsed, and that inflates the bound for the mid-stack. Repeating the solve with X from the *unfolded* run drops the ceiling from 0.925 to 0.778 at $il=5$ and from 0.933 to 0.738 at $il=24$ — three and a half times more energy reachable — while leaving $il=41$ at 0.951. Since any repair is applied to every folded layer at once and therefore partially restores the trajectory the next layer sees, the honest statement for the mid-stack is a bracket: the joint-repair ceiling lies between 0.74 and 0.93, and we measure only the pessimistic end. We first reported this bracket for the mid-stack and claimed the deep layers escaped it, on the strength of $il=41$ sitting at 0.951 on both trajectories. Extending the solve to the other deep folded layers refutes that: $il=42$ — one layer deeper, sharing the same physical layer 40 — reaches 0.810 on the unfolded trajectory, and $il=44$ reaches 0.878, against 0.951 at $il=41$. Two independent nulls at $il=42$ (shuffled inputs, and a distant physical layer) both land on the chance floor, so 0.810 is a real basis match and not a solver artifact. $il=41$ is therefore the most

pessimistic layer we measured rather than a representative one, and there is no deep-layer exception: on the unfolded trajectory the whole folded stack brackets 0.74–0.95.

Re-pairing does not recover it either. Sweeping wl for logical layer 24 across distances 1 to 16 moves the ceiling only from 0.923 to 0.957, monotonically in distance (0.923 at $wl=25$, 0.928 at 23, 0.933–0.935 at distance 2, 0.945 at distance 4, and exactly the chance floor, 0.957, at distances 12 and 16). The deployed map uses distances 1 and 2, and the sweep's best point $wl=25$ is a *later* layer that a causal fold map cannot use; among usable pairings the best is 0.928. So the fold map here is already near the best available *re-pairing* at fixed R — which is not the same as saying fold-map search is useless, since changing R or pinning layers demonstrably does work (§4.5(ii), and $R=2$ scores 21.8 against $R=3$'s 27.6).

What this closes is one specific family: any repair confined to the *gating side* at fixed input — an oracle router, a refitted router, a gating adapter, a scalar or vector gain on the routed branch — cannot make the folded layer reproduce the unfolded layer's routed output, because the target lies almost entirely outside the reachable set. It does not close repairs that move the basis or the target rather than the mixing coefficients: an adapter on the always-on shared-expert path changes R itself and is outside the frame, and an input-side modulation feeds the experts a different x , which SwiGLU makes nonlinear in a way the span argument does not cover. We tested the one natural input-side choice, rescaling by the ratio of the two layers' FFN norms, and it buys nothing (0.9293 against 0.9292) — but that is one negative datum, not a closed door.

One further extrapolation must be refused, and our own data are what refuse it: the bound does not say that no gating-side change can lower *perplexity*. The per-layer objective it bounds and the end-to-end objective are anti-correlated in exactly the region that matters. Fitting γ per layer to minimize the per-layer residual drives it toward a median of 0.15, and that setting scores 40.8 end-to-end, while $\gamma = 0.5$ — far from the per-layer optimum — scores 27.6. Driving per-layer reconstruction error down moves perplexity the wrong way by roughly 50%. That is a caution for layer-wise repair generally, since the GPTQ/SparseGPT framing takes per-layer reconstruction as the objective; here it is the wrong one.

What survives is a mechanistic statement about the basis, not an impossibility proof. On the trajectory the fold actually produces, layer wl 's expert basis does not contain layer il 's routed function — so a repair confined to the gating side of the executed fold has very little to work with. That is consistent with why loop conversion [3] is priced in tens of billions of tokens rather than in calibration batches: the procedure is building the shared basis, not fitting a correction on top of an adequate one. Read with §4.2, the two results delimit the contribution. Folding is a storage-residency mechanism, and on that axis it now delivers GPU decode of a

118B checkpoint at 34.4 t/s on a 32 GB laptop; this measurement gives no reason to expect it can also become a post-hoc compression method for checkpoints that were not trained to loop.

Scope of the bound. One checkpoint, three layers, 128 tokens drawn consecutively from one document, FFN-only folding with frozen experts. The least-squares relaxation makes it a lower bound rather than a tight estimate — the true constrained optimum is no better. It bounds per-layer output reconstruction by gating-side changes at fixed input; it does not bound end-to-end loss, and the γ result above shows those two are not interchangeable. It says nothing about what full-model training reaches, nothing about architectures whose layers are trained to share an expert basis, and nothing about folding applied to the mixer, which we did not analyze this way. It should be replicated on a second looped-MoE checkpoint before being read as general.

5 Discussion and limitations

Scale. Quality results are established at 15M (dense) and 42M (MoE) parameters on TinyStories — the dense arms' 30.4 MB is a resident-bytes figure, not a parameter count; transfer is supported by the literature at 0.1–8B [2,3,6] and by Nanbeige at 3B, but our own numbers claim nothing beyond their scale. Our token budgets are conservative for loops, which reportedly benefit from long training [6]; our recovery figures may therefore be underestimates. TinyStories cuts the other way: it is knowledge-light and syntactically regular — the regime loops handle best — so the same figures may be dataset-flattered; the net direction of the two biases is unmeasured. **Knowledge ceiling.** Saturation at $\sim$ half the gap is consistent with loops buying depth, not storage [15]; gains concentrate on reasoning-flavored ability, and knowledge-heavy use needs the parameter/retrieval axis. **KV cache and context length.** Folding shrinks resident weights but not the KV cache, which scales with logical depth and context length regardless of R. Figure 4 makes the regime map explicit for Llama-3.3-70B (80 layers, GQA 8 kv-heads, head dim 128; 320 KiB/token at fp16): under a 24 GB usable budget and counting the same $\sim$ 1 GB of runtime buffers §4.1 budgets, R=2 (21.98 GB weights) sustains only $\sim$ 3k tokens of context; R=3 reaches $\sim$ 24k; R=4 $\sim$ 34k; KV at q8 extends R=2 to $\sim$ 6k. Cross-loop KV sharing [7] is therefore not optional polish: for any deployment wanting both a low fold factor (better recovered quality, §3.4) and long context on fixed RAM, it is a condition of validity. We did not evaluate it and flag it as the largest open item between §3's quality results and a long-context deployment of §4. **Systems.** Measurements are one device (M1 Max, one NVMe class). The Metal path, open in v2.2, now runs (§4.2), but the cliff rows remain CPU-backend and we did not re-measure the streamed baselines on Metal, so every multiplier reported here is a CPU multiplier; the Metal row also rests on a workaround for an upstream scheduler defect (Appendix A(8)) rather than on a fix we

upstreamed, and it covers decode only — batched prefill at R=3 crosses this device's working-set limit and returns NaN (§4.2, Appendix A(11)), so we have shown GPU decode of a folded 118B checkpoint, not an end-to-end serving path.

Folded quality. v2.2 sacrificed it by choice; v3 bounds it by measurement (§4.5), and the bound is the more useful statement because it rules out an entire class of cheap repairs rather than deferring them. The practical deployment story therefore requires loop-trained or loop-converted [3] checkpoints, whose conversion cost (tens of B tokens) we did not pay — §4.5 is our account of why that cost is not avoidable by calibration. The bound itself is single-checkpoint, is measured on the folded model's own collapsed trajectory (which inflates it for the mid-stack), and constrains gating-side repair rather than training-free repair in general.

Claim discipline. All kill-gates were fixed pre-run; the novelty claim was adversarially searched before writing and is stated, with its framing risk acknowledged, in §2. This revision incorporates an adversarial review: the multi-seed replication (§3.3), the compute-matched control (§3.2), the quantization baseline (§4.2), and the flattened twin (§4.4) each exist because the review demanded them, and two headline claims of the first draft ("92%" and the MoE 2.7 $\times$) were retired or corrected by the added measurements. A post-v2 cross-check by independent AI review sessions (v2.1; see the authorship footnote) added the 2024–2026 looped-MoE line [18–21] that the original novelty search had missed, and the token-axis/loop-axis sticky disambiguation of §3.3. v3 was likewise revised after a four-lens adversarial review, which is why §4.5 reports a chance floor and a second trajectory it did not originally have, and why the Metal prefill failure is reported as an ordering rather than a diagnosis; the review also refuted the first draft's reading of its own bound (a relative residual of 0.93 corresponds to 30–37% of the target norm, not the 5–7% first written). For completeness, v3 also corrects three v2.2 numbers that were revised without an erratum note in the first v3 draft: folded 70B weights 21.2 $\rightarrow$ 21.98 GB (summed per tensor rather than file size / R, which lowers every Fig. 4 context ceiling — R=2 from $\sim$ 9k to $\sim$ 6k), the looped-block expert-touch figure 2.37 $\rightarrow$ 2.74 (Table 2's 2.369 is the four-module average, and only 2.74 is consistent with the $\sim$ 81% self-consistency figure), and Table 1's *large* resident weights 55.9 $\rightarrow$ 56.1 MB. The §1 router-coverage figure is corrected from 98% to the 75% the published census supports.

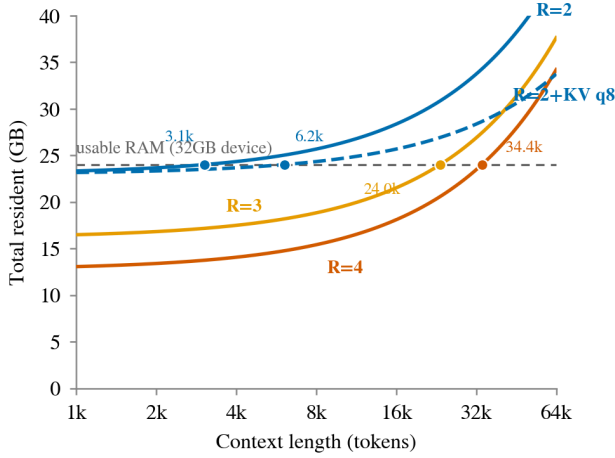


Figure 4. Total resident memory (folded weights + KV cache) vs context length for Llama-3.3-70B at $R=2/3/4$ (fp16 KV) and $R=2$ with q8 KV. Markers show the maximum context each configuration sustains under a 24 GB usable-RAM budget on a 32 GB device. The curves include the ~ 1 GB of runtime buffers §4.1 budgets alongside weights and KV; omitting that term (as this figure did before v3) overstates every ceiling, and after the $21.2 \rightarrow 21.98$ GB weight correction it would nearly double $R=2$'s. The map is analytic (folded weights + per-token KV arithmetic, all in decimal GB), not measured; folded weights are summed per tensor from the GGUF and include the non-block tensors, which never fold, and exact folding requires K to divide the layer count: at 80 layers $R=2$ ($K=40$) and $R=4$ ($K=20$) are realizable, while the $R=3$ curve ($K \approx 26.7$) is an idealized interpolation shown for the trend. Table 3's folded rows use $K=40$ (llama, $R=2$) and $K=16$ (laguna, 48 layers, $R=3$).

6 Conclusion

Treated as a residency optimization, recurrent depth turns the fold factor into a deployment knob: train at $R=3-4$ (where returns are still rich), ship the folded stack, execute at logical depth, and route sticky on bandwidth-bound hardware — a policy the multi-seed data shows costs nothing. On a 32 GB laptop this converts a storage-cliff regime measured in hundredths of a token per second into RAM-speed decoding, and with the loader scoped to the folded working set it reaches the GPU: a 57.6 GB checkpoint decodes at 34.4 tokens/s on a machine that cannot hold it. The flattened-twin experiment pins the conversion on residency alone — same function, same output, $\sim 150\times$ the speed.

What the fold factor does *not* buy is quality on weights that were never trained to share. We measured how far that can be pushed without training and where it stops: on the trajectory the fold produces, the physical layer's expert basis is close to a random subspace for its neighbour's routed function, and no gating-side change reaches past that. That is the sense in which this is a deployment mechanism for loop-native and loop-converted checkpoints rather than a compression trick for arbitrary ones — and today the checkpoints that satisfy it are the §4.3/§4.4 class, not the 70B and 118B rows we folded to isolate the physics. The ceiling result also carries a caution beyond folding: the per-layer reconstruction objective that layer-wise repair methods optimize is, here, anti-correlated with end-to-end loss. The missing half of the quality gap

points to composing loops with knowledge-axis techniques (memory layers, retrieval), which we leave to future work.

† The literature survey, implementation, experiment orchestration and overnight operation, and — in v3 — the diagnosis of the backend scheduler defect (Appendix A(8)) and the design of the achievability-ceiling measurement (§4.5) were carried out with AI research assistants (Claude, Anthropic Fable 5 and Opus 5), including the adversarial reviews whose findings this revision incorporates; research direction, acceptance or rejection of every result, resources, and responsibility for the content rest with the human author. All measurements are reproducible from the author's logs and the public repository (github.com/cornch-k/fold-dont-stream); total compute cost was one \$9.99 GPU-credit purchase plus a consumer laptop. This document is a working preprint; it has not been peer reviewed.

References

- [1] M. Dehghani et al. Universal Transformers. arXiv:1807.03819, 2018.
- [2] Z. Liu et al. MobileLLM: Optimizing Sub-billion Parameter Language Models for On-Device Use Cases. ICML 2024. arXiv:2402.14905.
- [3] S. Bae et al. Relaxed Recursive Transformers. arXiv:2410.20672, 2024.
- [4] J. Geiping et al. Scaling up Test-Time Compute with Latent Reasoning: A Recurrent Depth Approach. arXiv:2502.05171, 2025.
- [5] S. Bae et al. Mixture-of-Recursions. NeurIPS 2025. arXiv:2507.10524.
- [6] ByteDance. Ouro: Looped Language Models. arXiv:2510.25741, 2025.
- [7] V. Conchello Vendrell et al. Memory-Efficient Looped Transformer: Decoupling Compute from Memory in Looped Language Models (MELT). arXiv:2605.07721, 2026.
- [8] M. Jaggi. Tying the Loop — Tied Expert Layers in Mixture-of-Experts Language Models. arXiv:2606.16825, 2026.
- [9] K. Alizadeh et al. LLM in a Flash: Efficient LLM Inference with Limited Memory. arXiv:2312.11514, 2023.
- [10] Y. Sheng et al. FlexGen: High-Throughput Generative Inference of LLMs with a Single GPU. ICML 2023.
- [11] MoE-Infinity: Offloading-Efficient MoE Model Serving. arXiv:2401.14361, 2024.
- [12] A. Kayyam. Sticky Routing: Training MoE Models for Memory-Efficient Inference. arXiv:2607.08780, 2026; X. Zhu et al. ReMoE: Boosting Expert Reuse through Router Fine-Tuning in Memory-Constrained MoE LLM Inference. ICML 2026. arXiv:2605.27081.
- [13] SHARP: Sharing Adjacent Layers for Efficient On-Device Inference. arXiv:2502.07832, 2025.
- [14] llama.cpp discussion #4718: in-situ auto-frankenmerges (unimplemented proposal), 2024.
- [15] Z. Allen-Zhu, Y. Li. Physics of Language Models, Part 3.3: Knowledge Capacity Scaling Laws. arXiv:2404.05405, 2024.
- [16] Nanbeige. Nanbeige4.2-3B: a Looped-Transformer agent model. HuggingFace model card, 2026.
- [17] llama.cpp PR #25994: Add support for Nanbeige4.2 (looped transformer), 2026.
- [18] R. Csordás, K. Irie, J. Schmidhuber, C. Potts, C. D. Manning. MoEUT: Mixture-of-Experts Universal Transformers. NeurIPS 2024. arXiv:2405.16039.
- [19] W. Chen et al. LoopMoE: Unifying Iterative Computation with Mixture-of-Experts for Language Modeling. arXiv:2606.04438, 2026.
- [20] A. Zeitoun, L. Torroba-Hennigen, Y. Kim. Hyperloop Transformers. arXiv:2604.21254, 2026.
- [21] C. Deng et al. LT2: Linear-Time Looped Transformers. arXiv:2605.20670, 2026.

Appendix A Measurement hygiene on consumer hardware

Eleven lessons are recorded so others skip them; the first five consumed one evening. (1) `llama-bench` cannot disable ARM weight repacking; repacking a 42.5 GB model allocates comparable anonymous memory, OOM-killing the run — use `llama-cli --no-repack`. (2) The current `llama-cli` TUI does not exit after `-n` tokens even with conversation mode off; a log-watching watchdog that kills on the performance line is required. (3) Default load-time prefetch touches the whole file and destroys residency observations; disable it. (4) On Apple Silicon, offloading a folded model to Metal registered *all* mapped tensors as wired GPU memory (42.5 GB wired on a 32 GB machine), freezing the system while process RSS read 1 GB — RSS- and swap-based guards are blind to it. Resolved in v3 (§4.2) by skipping tensor creation for weights the folded graph never reads and copying the rest into device-private buffers per tensor; wired memory then tracks the folded working set. The blindness itself generalizes beyond Metal: mmap-backed weight pages are file-backed and are charged to neither RSS nor swap, so a process can hold tens of GB resident while Activity Monitor reports hundreds of MB. On macOS the only honest residency signal is `vm_stat` page-in accounting, which is what every SSD-read column in this paper reports. (5) A swap-delta guard (abort beyond +1.5 GB) and per-run page-in accounting made every reported run verifiably swap-free. (6) Single-run throughput on mmap-backed models is unreliable: an early Laguna fold read 3.8 t/s that $n=3$ repetition corrected to 2.37 ± 0.42 — the OS page cache is a hidden variable across runs; report means over repetitions. (7) For the twin experiment (§4.4), RAM was constrained with an `mlock` ballast process rather than a VM or `cgroup`; the unconstrained control (both twins at an identical 15.6 t/s) doubles as the check that the constraint, not the flattening itself, causes the collapse. (8) `ggml`'s backend scheduler copies MoE expert stacks to the accelerator on demand but memoizes each copy globally by (tensor, backend, slot) rather than per graph split. When a later split references a stack an earlier split already "copied", no copy is issued and the kernel reads never-written bytes of an uninitialized private buffer — NaN logits, no error, no warning. It reproduces without folding, and is the same class

as an upstream fix that was applied to the routing *ids* but not to the weight operand. Disabling op offload avoids the path and restores the CPU value to four decimals (1307.5595) — but those runs are at `-ngl 0`, where that switch also removes Metal from the computation entirely, so the agreement is a workaround and not a confirmation of the mechanism. The confirming test, forcing the full-tensor copy while op offload stays enabled, we did not run, and we have not written the standalone reproducer that would demonstrate the fold-independence we infer from the code. (9) When dumping activations to analyze a folded model layer by layer, key each dump by the *logical* layer — the activation tensor — and never by the weight tensor: folded layers share weights, so weight-keyed dumps silently merge several logical layers into one file and every downstream number is quietly wrong. We caught this only because an unfolded layer reported a cosine of 0.449 against itself, which is impossible; it is the reason each comparison script here carries a self-check whose expected value is known before the run. (10) Environment-variable knobs parsed with `atoi/atof` read an empty string as 0. A shell quoting slip that passed an empty γ therefore set $\gamma = 0$, deleting the routed branch entirely, and the run completed with a plausible perplexity that we nearly recorded. Reject the empty string explicitly rather than trusting the parse. (11) Metal reports a recommended `MaxWorkingSetSize` well below installed RAM (26.8 GB of 34.4 GB here, with `iogpu.wired_limit_mb` at its default). Crossing it does not fail uniformly: a load that overshoots by several GB raises `kIOGPUCommandBufferCallbackErrorOutOfMemory`, but one that overshoots narrowly — weights inside the limit, prefill compute buffers outside it — returns NaN with no error at all, while the batch-1 decode path on the same weights stays numerically correct. Sweeping the fold factor is what surfaced the ordering; a batch sweep alone would have suggested a kernel bug. Note the sweep is not a clean single-variable experiment — changing `R` changes the fold map, the unique-block count and the set of tensors the loader skips as well as the byte total — which is why §4.2 reports it as an ordering rather than a diagnosis. The one pair that does isolate size is `R=3` with and without the shared-expert knob: 0.4 GB apart, one finite and one NaN.